

## 第 48 讲核心概念速查卡：CNN 基础

### 1. 图像 Tensor

PyTorch 图像常用：

$$(N, C, H, W)$$

$N$ =Batch,  $C$ =Channel,  $H$ =Height,  $W$ =Width。

### 2. 卷积的最小本质

一个输出值：局部 **Patch** 与 **Kernel** 对应元素相乘再求和。

同一个 Kernel 滑过所有空间位置  $\Rightarrow$  Parameter Sharing。

输出的整张响应矩阵  $\Rightarrow$  Feature Map。

### 3. Shape 公式

Dilation=1 时：

$$H_{out} = \left\lfloor \frac{H + 2P - K}{S} \right\rfloor + 1$$

Width 同理。

### 4. Conv2d Weight Shape

$$W : (C_{out}, C_{in}, K_h, K_w)$$

$$b : (C_{out},)$$

$$(N, C_{in}, H, W) \rightarrow (N, C_{out}, H_{out}, W_{out})$$

参数量：

$$C_{out}C_{in}K_hK_w + C_{out}$$

### 5. Pooling / 平移

MaxPool：局部窗口取最大值，通常无可学习参数。

Convolution 更接近 translation **equivariance**：输入移动，Feature Map 响应也移动；不等于自动获得完全 translation invariance。

### 6. 三个高频误区

- PyTorch Conv2d 严格数学上使用 cross-correlation，不翻转 Kernel。
- Kernel 不是人手固定的边缘算子；训练中通常通过 Gradient 学出来。
- 参数共享减少的是 Parameter 数，不代表计算量与输入 H/W 无关。

### 7. TinyCNN Shape 记忆

$$(N, 1, 8, 8) \xrightarrow{\text{Conv}(1 \rightarrow 4)} (N, 4, 8, 8) \xrightarrow{\text{Pool}2} (N, 4, 4, 4) \xrightarrow{\text{Flatten}} (N, 64) \xrightarrow{\text{Linear}} (N, 2)$$

### 8. CNN vs Transformer

CNN: 固定局部 Kernel + 空间参数共享 + 强局部归纳偏置。

Transformer: Attention 根据内容动态决定 Token/Patch 间交互, 长距离关系更直接。

### 9. 科研读图先问

输入 Shape?  $C_{in}/C_{out}$ ? Kernel/Stride/Padding? 空间尺寸在哪里下降? 参数量来自哪里? 是否有 Pooling / Residual / Norm? 比较是否控制参数量与训练预算?

### 10. 与前面课程连接

- 卷积 = 局部乘加, 本质仍是线性运算。
- ReLU = 非线性。
- Shared Parameter 的 Gradient = 多个空间路径贡献累加。
- 训练流程仍是 Forward  $\rightarrow$  Loss  $\rightarrow$  Backward  $\rightarrow$  Optimizer。